

Individual Project Report - UCAB Platform

Full-Stack Developer & Documentation Specialist Contribution

1. Project Metadata

Project Title: UCAB - Full-Stack Cab Booking & Ride-Hailing Platform

Author: Konatham Sreevanth

My Role: Full-stack Developer and Documentation Specialist

Team Collaborators: Niharika Devi Guttula, Madiki Devi, Mahipala Veera Venkata Manikanta, and Naveen Kumar Kondepudi

2. Executive Summary & Project Scope

The UCAB application is a web-based, full-stack cab booking and taxi dispatch software designed to streamline passenger-driver connectivity. The platform manages JWT authentication, user and driver dashboards, simulated payment processing, and customer support channels. As a Full-stack Developer and Documentation Specialist on the team, my primary objective was to coordinate full-stack feature development, implement test-verification pipelines, ensure API compatibility, and compile the technical layout of the system.

3. My Key Contributions & Work Accomplished

- **Full-Stack Feature Integration:** Collaborated on frontend React route setups and backend controllers to connect ride-status updates and payment simulation APIs. Managed state sync between passenger ride requests and driver dashboards.
- **System Quality Assurance & Integration Testing:** Created and executed system-wide integration test scenarios. Verified authorization controls on restricted API endpoints and tested ride matching sequences (Requested -> Accepted -> Arrived -> Started -> Completed).
- **Error Diagnosis & Code Debugging:** Identified and debugged rendering loops in the React state machine, resolved driver online/offline toggling bugs, and corrected database model validation checks in the Express backend.
- **Technical Documentation Compilation:** Architected the project documentation including database model schemas, API endpoint specifications, and local installation manuals to ensure developer onboarding is clean.

4. System Architecture Overview

Frontend Stack: Built with React.js (Vite framework) with responsive styled dashboards, centralized contexts for global auth state, and Axios for handling API communications.

Backend Stack: Node.js and Express.js REST APIs utilizing custom middleware for JWT generation, route guarding, and unified error handling.

Database Layer: MongoDB with Mongoose schemas for User, Driver, Ride, Payment, and Support models, running on MongoDB Memory Server for isolated testing.

5. Local Environment Setup & Run Procedures

Setup Phase: Install Node.js (v18+) and execute 'npm run install-all' from the project root to configure dependencies in both directories. Manage local configuration using backend environment parameters.

Execution Phase: Run 'npm run dev' to concurrently launch the Express REST API and Vite server. Access the dashboards on local ports and test authentication flows.

6. Specific Verification Scenarios Completed

User Authentication: Validated password hashing on registration, JWT token generation upon login, and route protections for drivers and administrators.

Ride Booking Workflow: Created mock ride bookings from passenger dashboards, checked database insertions, verified availability changes, and updated trip statuses to completion.

Simulated Payment & Support: Tested fare collection triggers and simulated payment state updates; validated support ticketing logs for customer queries.

7. Future Goals & Roadmap

For future releases, my focus will be on integrating the Google Maps SDK for real-time map plots and live GPS routing, implementing WebSockets (Socket.io) for live coordination, upgrading mock payments to Stripe checkout elements, and wrapping the dashboards in a React Native codebase.